

1. What is the difference between a class and an object?

A class is a blueprint or template that defines the structure and behavior of objects. An object is an actual instance of a class that represents a real entity in memory.

2. What happens if you define a parameterized constructor in a class?

When a parameterized constructor is defined, Java does not automatically create a default no-argument constructor. If a no-argument constructor is still required, it must be explicitly written by the programmer.

3. List Java access modifiers from most restrictive to least restrictive.

The access modifiers in Java from most restrictive to least restrictive are: private, default (package-private), protected, and public.

4. What is the difference between method overloading and method overriding?

Method overloading occurs when multiple methods share the same name but have different parameter lists and is resolved at compile time. Method overriding occurs when a subclass provides a specific implementation of a method defined in its parent class and is resolved at runtime.

5. Can a final method or a private method be overridden? Why or why not?

A final method cannot be overridden because it is intended to prevent modification of its implementation. A private method cannot be overridden because it is not visible to subclasses and therefore does not participate in inheritance.

6. When does static polymorphism and dynamic polymorphism occur?

Static polymorphism occurs at compile time and is achieved through method overloading. Dynamic polymorphism occurs at runtime and is achieved through method overriding and inheritance.

7. Why does Java not support multiple inheritance of classes? How is this limitation handled?

Java does not support multiple inheritance of classes to avoid ambiguity such as the diamond problem. This limitation is handled by allowing a class to implement multiple interfaces.

8. What OOP principle is demonstrated by: `List list = new ArrayList<>();`

This demonstrates polymorphism and programming to an interface. The reference variable uses the interface type while the actual object is an implementation class.